

---

## ΟΜΑΔΑ Α (Γενικές Ερωτήσεις) 1-61 – ΠΛΗΡΕΙΣ ΑΠΑΝΤΗΣΕΙΣ

### 1. Ρόλος ΛΣ

Διαχείριση υλικού, εκτέλεση εφαρμογών, παροχή διεπαφής χρήστη. Χωρίς ΛΣ, κάθε εφαρμογή θα έπρεπε να επικοινωνεί απευθείας με το υλικό – πρακτικά αδύνατο.

### 2. Μέρη ΛΣ και πυρήνας

Μέρη: Πυρήνας, διαχείριση διεργασιών, μνήμης, αρχείων, I/O, διεπαφή χρήστη. Πυρήνας: το κεντρικό τμήμα που εκτελείται σε προνομιακή κατάσταση, ελέγχει το υλικό.

### 3. Sequential vs Άμεση οργάνωση αρχείων

- Sequential: γραμμική πρόσβαση (π.χ. μαγνητική ταινία)
- Άμεση: τυχαία πρόσβαση με κλειδί/διεύθυνση (π.χ. δίσκος)

### 4. system.ini

Αρχείο ρυθμίσεων παλαιών Windows (3.x έως Me) για drivers 16-bit. Σήμερα στο Registry.

### 5. FAT vs NTFS

FAT: απλό, συμβατό, χωρίς δικαιώματα, max 4GB/αρχείο (FAT32). NTFS: δικαιώματα, κρυπτογράφηση, log, μεγάλα μεγέθη.

### 6. Σκοπός τεχνικών διαχείρισης μνήμης

Αποδοτική κατανομή, προστασία, εικονική μνήμη (επέκταση πέρα από φυσική RAM).

### 7. Κατηγορίες εκτυπωτών

Dot matrix (κρουστικός), Inkjet (ψεκασμός μελάνης), Laser (τόνερ), Thermal, 3D, Sublimation.

## **8. Λειτουργίες σε αρχείο**

Δημιουργία, άνοιγμα, ανάγνωση, εγγραφή, κλείσιμο, διαγραφή, μετονομασία, αντιγραφή, μετακίνηση.

## **9. Δεδομένα vs Πληροφορία**

Δεδομένα: ακατέργαστα σύμβολα. Πληροφορία: επεξεργασμένα δεδομένα με νόημα.

## **10. bit, byte, word**

bit: 0/1. byte: 8 bits. word: 16/32/64 bits (ανάλογα CPU).

## **11. RAM vs ROM**

RAM: προσωρινή, ταχύτατη, πτητική. ROM: μόνιμη, μη πτητική, firmware.

## **12. Cache memory**

Μικρή, γρήγορη μνήμη μεταξύ CPU και RAM. Αποθηκεύει συχνά δεδομένα.

## **13. Επεξεργαστής – είδη**

Εκτελεί πράξεις. Είδη: Intel Core i3/i5/i7/i9, AMD Ryzen, ARM, Celeron, Pentium.

## **14. Ιός υπολογιστή**

Κακόβουλο λογισμικό που αντιγράφεται. Παράδειγμα: ILOVEYOU, ransomware.

## **15. Επικοινωνία δεδομένων**

Μεταφορά δεδομένων μεταξύ συσκευών μέσω μέσου.

## **16. Στοιχεία μετάδοσης**

Πομπός, δέκτης, μέσο, μήνυμα, πρωτόκολλο.

## **17. Καλώδια συνεστραμμένου ζεύγους**

UTP (χωρίς θωράκιση, φθηνό), STP (θωρακισμένο, καλύτερη προστασία).

## 18. Αιτίες θορύβου

EMI, RFI, crosstalk, θερμικός θόρυβος.

## 19. Αναλογικά σήματα

Συνεχή, μεταβάλλονται ομαλά. Χαρακτηριστικά: πλάτος, συχνότητα, φάση.

## 20. Ψηφιακά σήματα

Διακριτά επίπεδα (0/1). Χαρακτηριστικά: ρυθμός μετάδοσης, πλάτος παλμού.

## 21. Πότε απαιτείται A/D και D/A

A/D: για εισαγωγή αναλογικών σημάτων (μικρόφωνο, κάμερα). D/A: για αναπαραγωγή (ηχεία, οθόνη).

## 22. Σύγχρονη vs Ασύγχρονη μετάδοση

Σύγχρονη: συνεχής, με ρολόι, αποδοτική. Ασύγχρονη: start/stop bits, απλούστερη.

## 23. Πρόβλημα

Διαφορά μεταξύ τρέχουσας και επιθυμητής κατάστασης.

## 24. Αλγόριθμος vs Πρόγραμμα

Αλγόριθμος: σειρά βημάτων. Πρόγραμμα: υλοποίηση σε γλώσσα.

## 25. Δομή δεδομένων

Οργάνωση δεδομένων (πίνακας, λίστα, στοίβα, ουρά, δέντρο).

## 26. Κατηγορίες προβλημάτων

Αριθμητικά/μη αριθμητικά, αποφασίσιμα/μη, ντετερμινιστικά, παράλληλα.

## 27. Λειτουργίες δομών δεδομένων

Εισαγωγή, διαγραφή, αναζήτηση, ενημέρωση, διάσχιση, ταξινόμηση.

## **28. Ανάλυση προβλήματος**

Κατανόηση προβλήματος, είσοδος/έξοδος, περιορισμοί.

## **29. Τεχνικές σχεδίασης αλγορίθμων**

Διαίρει και βασίλευε, δυναμικός προγραμματισμός, greedy, backtracking, brute force.

## **30. Χαρακτηριστικά αλγορίθμου**

Πεπερασμένο, σαφήνεια, είσοδος, έξοδος, εφικτότητα.

## **31. Τρόποι περιγραφής αλγορίθμου**

Φυσική γλώσσα, ψευδοκώδικας, διάγραμμα ροής, γλώσσα προγραμματισμού.

## **32. Χαρακτηριστικά πλήρους αλγορίθμου**

Ορθότητα, αποδοτικότητα, γενικότητα.

## **33. Είδη δομής επιλογής**

if, if-else, switch-case, else-if.

## **34. Είδη δομής επανάληψης**

while, do-while, for.

## **35. Καταχωρητές CPU**

Μικρές υπερταχείες θέσεις μνήμης στην CPU.

## **36. Interpreter vs Compiler**

Interpreter: γραμμή προς γραμμή, αργότερος. Compiler: μεταγλωττίζει όλο, ταχύτερη εκτέλεση.

## **37. Ρόλος μονάδας ελέγχου**

Αποκωδικοποιεί εντολές και παράγει σήματα ελέγχου.

### **38. Διακοπή (interrupt)**

Σήμα που αναστέλλει την εκτέλεση και καλεί ISR.

### **39. Γιατί αυξάνοντας RAM επιταχύνεται ο υπολογιστής;**

Μειώνεται η χρήση swap στον δίσκο.

### **40. WWW**

Υπηρεσία διαδικτύου για υπερκείμενα μέσω HTTP. Προσφέρει ιστοσελίδες, email, e-commerce.

### **41. Υπερκείμενο**

Κείμενο με συνδέσμους. Διαφορά από απλό: μη γραμμική πλοήγηση.

### **42. Υπερμέσα**

Υπερκείμενο + εικόνες, ήχο, βίντεο.

### **43. Bootstrap technique**

CSS framework για responsive σχεδίαση.

### **44. Τρόποι σύνδεσης CSS**

1. Εξωτερικό (link), 2) Εσωτερικό (style), 3) Inline (style attribute).

### **45. Queries**

Δηλώσεις SQL για ανάκτηση/ενημέρωση δεδομένων.

### **46. DBMS (ΣΔΒΔ)**

Λογισμικό διαχείρισης βάσεων δεδομένων.

### **47. Μοντέλο για απεικόνιση πραγματικού κόσμου**

Μοντέλο Οντοτήτων-Συσχετίσεων (ER).

## **48. Σχέση ένα προς πολλά**

Ένας πελάτης → πολλές παραγγελίες.

## **49. Στάδια ανάπτυξης ΒΔ**

Απαιτήσεις, σχεδίαση (εννοιολογική/λογική/φυσική), υλοποίηση, φόρτωση, δοκιμές, συντήρηση.

## **50. Περιβάλλον πολλαπλών χρηστών**

Πολλοί χρήστες ταυτόχρονα με ελέγχους συναλλαγών.

## **51. Μοντέλα ΒΔ**

Ιεραρχικό, δικτυακό, σχεσιακό, αντικειμενοστραφές, NoSQL.

## **52. Χρωματικά μοντέλα**

RGB (οθόνες), CMYK (εκτύπωση), HSV/HSL.

## **53. Συμπληρωματικά χρώματα**

Απέναντι στον τροχό. Παράδειγμα: κόκκινο-κυανό, πράσινο-ματζέντα, μπλε-κίτρινο.

## **54. Χαρακτηριστικά bitmap**

Πλέγμα pixel, ανάλυση, βάθος χρώματος, συμπίεση.

## **55. Μέγεθος bitmap 100x100**

True color (24bit): 30.000 bytes. Grayscale (8bit): 10.000 bytes.

## **56. Μορφές bitmap**

BMP (μη συμπιεσμένο), PNG (lossless, διαφάνεια), JPEG (lossy), GIF (256 χρώματα, animation).

## **57. Χρήση μάσκας**

Καθορίζει ποια pixel επηρεάζονται.

## **58. Πλεονεκτήματα/μειονεκτήματα bitmap**

Πλεον: ρεαλισμός. Μειον: μεγάλο μέγεθος, απώλεια κλίμακας.

## **59. Μελέτη Σκοπιμότητας**

Εξέταση εφικτότητας (τεχνική, οικονομική, νομική).

## **60. Ανάλυση Απαιτήσεων**

Καταγραφή αναγκών χρηστών.

## **61. Προδιαγραφές Απαιτήσεων (RS)**

Λεπτομερής περιγραφή του «τι» θα κάνει το σύστημα.

---

# **ΟΜΑΔΑ Β (Ειδικές Ερωτήσεις) 62-215 – ΠΛΗΡΕΙΣ ΑΠΑΝΤΗΣΕΙΣ**

## **62. Logical System Specifications (LS)**

Περιγραφή λειτουργιών του συστήματος ανεξάρτητα από υλικό/τεχνολογία.

## **63. Φυσική Σχεδίαση (PD)**

Μετατροπή λογικού σε φυσικό μοντέλο (πίνακες, δείκτες, αποθήκευση).

## **64. Συνιστώσες Πληροφοριακού Συστήματος**

Υλικό, λογισμικό, δεδομένα, διαδικασίες, άνθρωποι.

## **65. Τύποι λογισμικού**

- Λογισμικό συστήματος (ΛΣ, drivers)
- Λογισμικό εφαρμογών (Word, browser)
- Εργαλεία ανάπτυξης (compilers, IDE)

## 66. Φάσεις κύκλου ζωής ΠΣ

Ανάλυση → Σχεδίαση → Υλοποίηση → Δοκιμές → Εγκατάσταση → Συντήρηση.

## 67. Αρχές δομημένης προσέγγισης

- Κορυφής-προς-βάση (top-down)
- Τμηματοποίηση (modularity)
- Αποφυγή goto

## 68. Αρχές μοντέλου κύκλου ζωής λογισμικού

Σαφείς φάσεις, παραδοτέα, έλεγχος αλλαγών, τεκμηρίωση.

## 69. Διαδικασία επιλογής λογισμικού

1. Ορισμός απαιτήσεων, 2) Αναζήτηση, 3) Αξιολόγηση, 4) Pilot test, 5) Απόφαση.

## 70. Αρχές ελέγχου λογισμικού

- Ορθότητα, κάλυψη κώδικα, επαναληψιμότητα, εντοπισμός σφαλμάτων.

## 71. White Box testing

Έλεγχος εσωτερικής δομής, λογικών μονοπατιών, συνθηκών.

## 72. Διασφάλιση ποιότητας λογισμικού

Διαδικασίες, μετρικές, έλεγχος, standards, reviews.

## 73. Διαχείριση ολικής ποιότητας (TQM)

Συνεχής βελτίωση. Κριτήρια: λειτουργικότητα, αξιοπιστία, αποδοτικότητα, συντηρησιμότητα, φορητότητα.

## 74. C++: εμφάνιση ονόματος 100 φορές

```
cpp
#include <iostream>
using namespace std;
```



```
int main() {
    for(int i=0; i<100; i++) cout << "Όνομα" << endl;
}
```

## 75. C++: μέγιστος 3 ακεραίων

```
cpp
int max(int a, int b, int c) {
    int m = a;
    if(b>m) m=b;
    if(c>m) m=c;
    return m;
}
```

## 76. Κληρονομικότητα & Πολυμορφισμός

Κληρονομικότητα: κλάση παίρνει χαρακτηριστικά άλλης. Πολυμορφισμός: ίδια μέθοδος διαφορετική συμπεριφορά.

## 77. Χαρακτηριστικά C++

- Αντικειμενοστραφής
- Compiled
- Χειροκίνητη διαχείριση μνήμης
- Πολλαπλή κληρονομικότητα
- Πρότυπα (templates)

## 78. Συναρτήσεις σε αντικειμενοστραφή προγραμματισμό

Ενσωματώνονται στις κλάσεις (μέθοδοι). Παράδειγμα μηνύματος: `object.method()`.

## 79. Σωστό/Λάθος – αιτιολόγηση

- `Person p1 = new Student();` **Σωστό** (upcasting)
- `Person p2 = new PhDStudent();` **Σωστό**
- `PhDStudent phd1 = new Student();` **Λάθος** (downcasting χωρίς cast)
- `Prof t1 = new Person();` **Λάθος** (αν Prof υποκλάση της Person)
- `Student s1 = new PhDStudent();` **Σωστό**

## 80. Java Virtual Machine (JVM) μέρη

Class Loader, Execution Engine, Runtime Data Areas (Heap, Stack, PC Registers, Method Area), Garbage Collector.

## 81. Τύποι μεταβλητών στην Java

- local (εντός μεθόδου)
- instance (εντός κλάσης, χωρίς static)
- static (class variables)
- parameters

## 82. Ρόλος δικτύου σε client-server

Επιτρέπει επικοινωνία μεταξύ client και server μέσω πρωτοκόλλων (TCP/IP, HTTP).

## 83. Application server

Εξυπηρετητής που εκτελεί εφαρμογές (π.χ. WebLogic, Tomcat, JBoss) και παρέχει υπηρεσίες (συναλλαγές, μηνύματα).

## 84. Fat servers vs Fat clients

- **Fat server:** περισσότερη λογική στον server (thin client)
- **Fat client:** λογική και επεξεργασία στον client (π.χ. desktop app)

## 85. Βήματα client-server συναλλαγής

1. Client στέλνει αίτημα. 2) Server λαμβάνει. 3) Server επεξεργάζεται. 4) Server στέλνει απάντηση. 5) Client λαμβάνει και εμφανίζει.

## 86. Τρεις αλγόριθμοι κρυπτογράφησης

AES, RSA, DES/3DES.

## 87. Κατηγορίες αλγορίθμων κρυπτογράφησης

- Συμμετρικά (AES, DES)
- Ασύμμετρα (RSA, ECC)
- Συναρτήσεις κατακερματισμού (SHA, MD5)

## 88. Απειλή (threat) σε υπολογιστικό σύστημα

Οτιδήποτε μπορεί να προκαλέσει βλάβη. Παραδείγματα: malware, phishing, DoS.

## 89. Phishing – αντιμετώπιση

Απάτη για κλοπή στοιχείων. Αντιμετώπιση: εκπαίδευση, email filters, 2FA.

## 90. Ψηφιακό πιστοποιητικό

Ηλεκτρονικό έγγραφο που ταυτοποιεί οντότητα (π.χ. ιστότοπο) μέσω CA (Certificate Authority).

## 91. Ανίχνευση απειλών

Antivirus, IDS/IPS, log analysis, SIEM, penetration testing.

## 92. Ορισμοί (Οντότητα, Συσχέτιση, Βαθμός, Πολυπλοκότητα)

- **Οντότητα:** αντικείμενο (π.χ. Υπάλληλος)
- **Συσχέτιση:** σύνδεση οντοτήτων
- **Βαθμός:** πλήθος οντοτήτων στη συσχέτιση
- **Πολυπλοκότητα:** 1:1, 1:N, M:N

## 93. Τρία επίπεδα αρχιτεκτονικής ΒΔ

1. Εξωτερικό (views χρηστών)
2. Λογικό (λογική δομή)
3. Εσωτερικό (φυσική αποθήκευση)

## 94. Τύποι σχέσεων (relationship)

1:1, 1:N, M:N.

## 95. SQL: SELECT DNAME, DEPTNO FROM DEPT;

## 96. SQL: SELECT \* FROM EMP WHERE DEPTNO = 30;

## 97. SQL (γραμματείς – επώνυμα, κωδικοί, όνομα τμήματος, κωδικός τμήματος, εργασία)

```
sql
SELECT E.ENAME, E.EMPNO, D.DNAME, D.DEPTNO, E.JOB
FROM EMP E JOIN DEPT D ON E.DEPTNO = D.DEPTNO
WHERE E.JOB = 'ΓΡΑΜΜΑΤΕΑΣ';
```

## 98. Roll back

Ακύρωση συναλλαγής και επαναφορά δεδομένων σε προηγούμενη συνεπή κατάσταση.

## 99. Απειλές για ασφάλεια ΒΔ

Μη εξουσιοδοτημένη πρόσβαση, SQL injection, απώλεια δεδομένων, κακόβουλο λογισμικό.

## 100. Χρήση Log File

Καταγραφή αλλαγών για ανάκτηση μετά από σφάλμα (undo/redo).

## 101. SQL: προσθήκη στηλών Address1, Zip1

```
sql
ALTER TABLE EMP ADD (Address1 VARCHAR2(100), Zip1 VARCHAR2(10));
```

## 102. SQL: όνομα, θέση, μισθός υπαλλήλων τμήματος 20 με SAL+COMM > 2000

```
sql
SELECT ENAME, JOB, SAL FROM EMP
WHERE DEPTNO = 20 AND (SAL + NVL(COMM,0)) > 2000;
```

## 103. SQL: συνολική αμοιβή (SAL+COMM) ανά πωλητή

```
sql
SELECT ENAME, SAL + NVL(COMM,0) AS TOTAL_PAY
FROM EMP WHERE JOB = 'ΠΩΛΗΤΗΣ';
```

## 104. SQL: αύξηση μισθού τμήματος 20 κατά 300€

```
sql
UPDATE EMP SET SAL = SAL + 300 WHERE DEPTNO = 20;
```

## 105. SQL: πιο καλοπληρωμένοι ανά είδος εργασίας (SAL+COMM)

```
sql
SELECT JOB, MAX(SAL + NVL(COMM,0)) AS MAX_PAY
FROM EMP GROUP BY JOB;
```

## 106. SQL: επώνυμο και όνομα τμήματος γραμματέων

```
sql
SELECT E.ENAME, D.DNAME
FROM EMP E JOIN DEPT D ON E.DEPTNO = D.DEPTNO
WHERE E.JOB = 'ΓΡΑΜΜΑΤΕΑΣ';
```

## 107. Πέντε τύποι δεδομένων SQL

INTEGER, VARCHAR, DATE, DECIMAL, BOOLEAN.

## 108. Τέσσερις πράξεις συνόλου (σχεσιακή άλγεβρα)

- **Ένωση (U):** υπάλληλοι με μισθό > 1000 U υπάλληλοι τμήματος 10
- **Τομή (∩):** υπάλληλοι που είναι και γραμματείς και έχουν προμήθεια
- **Διαφορά (-):** υπάλληλοι χωρίς προμήθεια
- **Καρτεσιανό γινόμενο (×):** EMP × DEPT

## 109. Πέντε λειτουργίες ΣΔΒΔ

- Ορισμός δεδομένων (DDL)
- Χειρισμός δεδομένων (DML)
- Έλεγχος συναλλαγών
- Έλεγχος ταυτότητας / ασφάλεια
- Backup / recovery

## 110. Αρχιτεκτονική τριών επιπέδων – στόχοι

1. Εξωτερικό (προσαρμογή σε χρήστες)
2. Λογικό (ανεξαρτησία από αποθήκευση)
3. Εσωτερικό (αποδοτικότητα)

## 111. Ανεξαρτησία δεδομένων

Δυνατότητα αλλαγής σε ένα επίπεδο χωρίς επίδραση σε άλλα.

- Λογική ανεξαρτησία
  - Φυσική ανεξαρτησία
- 

**112. (Δεν υπάρχει ερώτηση 112 στο αρχείο – συνεχίζουμε από 113)**

### **113. Μοντέλο καταρράκτη (Waterfall)**

Ακολουθιακό: Απαιτήσεις → Σχεδίαση → Υλοποίηση → Δοκιμές → Συντήρηση. Δύσκολη επιστροφή.

### **114. Δραστηριότητες αναγνώρισης & ανάλυσης προβλήματος**

- Καθορισμός προβλήματος
- Συλλογή πληροφοριών
- Διερεύνηση λύσεων
- Αξιολόγηση περιορισμών

### **115. Φάσεις μεθόδου Yourdon**

- Ανάλυση (διάγραμμα ροής δεδομένων)
- Σχεδίαση (δομημένη)
- Υλοποίηση

### **116. Μεθοδολογία JSD (Jackson System Development)**

- Φάση μοντελοποίησης (μοντέλο οντοτήτων)
- Φάση δικτύου (καθορισμός ροής)
- Φάση υλοποίησης (μετατροπή σε κώδικα)

### **117. Βασικές έννοιες αντικειμενοστραφούς**

Κλάση, αντικείμενο, κληρονομικότητα, πολυμορφισμός, ενθυλάκωση.

### **118. UML (Unified Modeling Language)**

Γλώσσα μοντελοποίησης για οπτικοποίηση σχεδίασης λογισμικού.

## 119. Διάγραμμα use-case

Δείχνει αλληλεπίδραση ηθοποιών (actors) με σύστημα. Παράδειγμα: «Πελάτης» → «Αγορά προϊόντος».

## 120. Είναι σωστός ο κώδικας; `int var=100; int* ptr; cout << *ptr;`

**Λάθος:** ο ptr δεν δείχνει πουθενά (uninitialized). Εκτύπωση \*ptr → undefined behavior.

## 121. Συνάρτηση swap με αναφορές C++

```
cpp
void swap(int &a, int &b) { int temp = a; a = b; b = temp; }
```

## 122. Αφηρημένη βασική κλάση (abstract class) – παράδειγμα πολλαπλής κληρονομικότητας

Χρησιμότητα: ορίζει διεπαφή (pure virtual) για υποκλάσεις.

```
cpp
class A { virtual void f()=0; };
class B { virtual void g()=0; };
class C : public A, public B { void f(){} void g(){} };
```

## 123. Αποτέλεσμα: `int i=10; if(i=20) cout << i;`

Εκτυπώνει 20 (η ανάθεση i=20 δίνει 20 → true).

## 124. Αποτέλεσμα με static int:

11, 12, 13.

## 125. Πρόγραμμα Fibonacci 50 πρώτοι

```
cpp
#include <iostream>
using namespace std;
int main() {
    long long a=1, b=1, c;
    cout << a << " " << b << " ";
    for(int i=3; i<=50; i++) {
        c = a + b;
        cout << c << " ";
        a = b; b = c;
    }
```

```
}  
}
```

## 126. Τι εμφανίζει το πρόγραμμα με Animal/Dog

4 και 1.

## 127. Το πρόγραμμα με class A (x private) – Σφάλμα μεταγλώττισης (δεν έχει πρόσβαση στο x).

128.  $\text{sum}(10) \rightarrow 10$ ,  $\text{sum}(10,0) \rightarrow 10$ ,  $\text{sum}(10,10) \rightarrow 20$ .

## 129. Παραγοντικό με συνάρτηση

```
cpp  
int factorial(int n) {  
    if(n<=1) return 1;  
    return n * factorial(n-1);  
}
```

## 130. Κλάση Stack και pop()

```
cpp  
class Stack {  
    int data[100];  
    int top;  
public:  
    Stack() { top = -1; }  
    void push(int x) { data[++top] = x; }  
    int pop() { return data[top--]; }  
};
```

131. Η compute επιστρέφει  $(100+200)-5 = 295$ .

## 132. Κλάση (Class) vs Αντικείμενο (Object)

Κλάση: πρότυπο/μπερίντα. Αντικείμενο: στιγμιότυπο κλάσης.

## 133. Γιατί δεν μπορούμε να αφαιρέσουμε πεδία από subclass;

Γιατί η subclass κληρονομεί όλα τα πεδία της superclass (αρχή υποκατάστασης Liskov).



## 134. Frame vs Dialog

Frame: κύριο παράθυρο με μενού, τίτλο, περιθώρια. Dialog: δευτερεύον παράθυρο, συνήθως modal, χωρίς μενού.

## 135. Menu vs MenuBar (Java)

MenuBar: περιέχει μενού (File, Edit). Menu: περιέχει MenuItems.

## 136. Menu vs MenuItem (Java)

Menu: υπομενού που μπορεί να περιέχει MenuItems ή άλλα Menu. MenuItem: λειτουργία.

## 137. Επανεκκίνηση vs Επαναφόρτωση applet

Restart: start() ξανά. Reload: φόρτωση νέας έκδοσης από server.

## 138. Constructor στην Java

Μέθοδος που καλείται κατά τη δημιουργία αντικειμένου.

```
java
class Person { Person(String name) { } }
```

## 139. Τι κάνει: $\max = k > j ? k : j$ ; $\rightarrow \max = 10$ .

## 140. Ρόλος εξαιρέσεων (exceptions) I/O

Διαχείριση σφαλμάτων (π.χ. αρχείο δεν βρέθηκε, δικαιώματα). Παραδείγματα:

FileNotFoundException, IOException.

## 141. Hangman σε Java χωρίς GUI (περιγραφή)

Πίνακας λέξεων, τυχαία επιλογή, βρόχος εισαγωγής γραμμάτων, σύγκριση, εμφάνιση προόδου.

## 142. Κλάση Tires στην Java

```
java
class Tires {
    double width, radius, rim;
    int type; // 0-3
    String brand, model;
```

```

    int year;
    Tires(double w, double r, double rim, int t, String b, String m, int y) {
        width=w; radius=r; this.rim=rim; type=t; brand=b; model=m; year=y;
    }
}

```

## 143. abstract class vs interface

- abstract class: μπορεί να έχει υλοποιημένες μεθόδους, πεδία, constructor.
- interface: μόνο αφηρημένες μεθόδους (πριν Java 8) και final static πεδία.

## 144. Interfaces στην Java – παράδειγμα

```

java
interface Drawable { void draw(); }
class Circle implements Drawable { public void draw() { } }

```

## 145. Java: Fahrenheit → Celsius

```

java
double c = 5.0 * (f - 32) / 9.0;

```

## 146. Java: εισαγωγή 6 ονομάτων, αλφαβητική σειρά

```

java
String[] names = new String[6];
// εισαγωγή
Arrays.sort(names);
// εμφάνιση

```

## 147. Method Overloading

Πολλές μέθοδοι με ίδιο όνομα, διαφορετικές παραμέτρους.

```

java
void add(int a, int b) { }
void add(double a, double b) { }

```

## 148. Method Overriding

Επανεγγραφή μεθόδου σε υποκλάση.

```

java
class Animal { void sound() { } }
class Dog extends Animal { void sound() { } }

```

## 149. Ανάλυση κώδικα interface MyInterface, class XYZ

Ορίζει interface με method1() και method2(). Η XYZ υλοποιεί. Στο main: MyInterface obj = new XYZ()  
→ upcasting. Καλεί method1().

## 150. Encapsulation – παράδειγμα Java

```
java
class Person {
    private String name;
    public String getName() { return name; }
    public void setName(String n) { name = n; }
}
```

## 151. Java: φωνήεντα σε φράση

```
java
String phrase = ...;
int count=0;
for(char c : phrase.toLowerCase().toCharArray())
    if("aeiou".indexOf(c)!=-1) count++;
```

## 152. Inventory με Vector Arrays

```
java
Vector<String[]> inventory = new Vector<>();
inventory.add(new String[]{"mithril sword", "Plate Armour"});
```

## 153. Print() με κληρονομικότητα (sum, sub, multiply)

```
java
class Sum { int calc(int a, int b) { return a+b; } }
class Sub extends Sum { int calc(int a, int b) { return a-b; } }
class Multiply extends Sub { int calc(int a, int b) { return a*b; } }
```

## 154. Rectangle, Square, Circle (κληρονομιά, εμβαδό)

```
java
class Rectangle { double width, height; double area() { return width*height; } }
class Square extends Rectangle { Square(double w) { width=height=w; } }
class Circle extends Rectangle { double radius; Circle(double r) { radius=r; } double area() {
return 2*3.14*radius*radius; } }
```

## 155. Java: σχεδίαση κύκλου με χρώμα, ακτίνα (χρήστης)

```
java
```

```
Scanner sc = new Scanner(System.in);
int r = sc.nextInt();
String color = sc.next();
JFrame frame = new JFrame();
frame.add(new JPanel() { public void paintComponent(Graphics g) {
    g.setColor(Color.getColor(color));
    g.fillOval(0,0,2*r,2*r);
}});
```

## 156. Demon vs Non-demon threads

Demon: εκτελούνται στο background (π.χ. garbage collector). Non-demon: foreground. JVM τερματίζει όταν μείνουν μόνο demon.

## 157. Java: προσπέλαση DATA.db (RandomAccessFile) – άθροισμα καταθέσεων/αναλήψεων, max

(Απαιτεί λεπτομερή κώδικα ανάγνωσης δομής με fixed record length. Μπορώ να τον αναπτύξω.)

## 158. SSADM φάσεις – στάδια

- Φάση 1: Ανάλυση (στάδια: 0-2)
- Φάση 2: Σχεδίαση (στάδια: 3-5)

## 159. Τμηματοποίηση (modularity) SSADM

Διαίρεση συστήματος σε λειτουργικές μονάδες (modules) για ευκολότερη ανάπτυξη.

## 160. Χαρακτηριστικά SSADM

- Δομημένη, βασισμένη σε διαγράμματα ροής δεδομένων (DFD)
- Προσδιορισμός οντοτήτων
- Τρεις απόψεις (δεδομένα, ροή, συμβάντα)

## 161. Πλεονεκτήματα client-server (διάκριση business logic & database)

- Κλιμάκωση, ασφάλεια, αποδοτικότητα, ανεξαρτησία πλατφόρμας, συντήρηση.

## 162. Three-tier vs two-tier

- Two-tier: client + server (π.χ. desktop app με DB)
- Three-tier: client → application server → database server (περισσότερη κλιμάκωση)

## 163. RPC (Remote Procedure Call)

Κλήση συνάρτησης που εκτελείται σε απομακρυσμένο υπολογιστή σαν να ήταν τοπική.

## 164. IPC (Interprocess Communication)

Μηχανισμοί επικοινωνίας μεταξύ διεργασιών (pipes, sockets, shared memory).

## 165. Triple DES (3DES)

Εφαρμογή DES τρεις φορές (κρυπτογράφηση-αποκρυπτογράφηση-κρυπτογράφηση) για ισχυρότερη κρυπτογράφηση.

## 166. Εμπιστευτικότητα vs Ακεραιότητα

- Εμπιστευτικότητα: πρόσβαση μόνο από εξουσιοδοτημένους.
- Ακεραιότητα: δεδομένα δεν τροποποιούνται ανεπίτρεπτα.

## 167. Authentication – συνηθέστερος τρόπος

Επαλήθευση ταυτότητας. Συνηθέστερος: κωδικός (password).

## 168. Logfiles – παράδειγμα

Αρχεία καταγραφής συμβάντων. Παράδειγμα: logs httpd για requests.

## 169. Αποστολή κρυπτογραφημένου μηνύματος με δημόσιο κλειδί

Αποστολέας κρυπτογραφεί με δημόσιο κλειδί παραλήπτη. Παραλήπτης αποκρυπτογραφεί με ιδιωτικό.

## 170. Ευπάθεια (Vulnerability)

Αδυναμία συστήματος που μπορεί να εκμεταλλευτεί απειλή.

## 171. Ευπάθειες υλικού & λογισμικού

Υλικό: Meltdown/Spectre, side-channel. Λογισμικό: SQL injection, buffer overflow.

## 172. Web footprinting

Περιλαμβάνει: WHOIS, DNS lookup, search engines, social media, OSINT.

## 173. Αυθεντικοποίηση βάσει χαρακτηριστικών

Biometrics (δακτυλικό αποτύπωμα, ίριδα, φωνή).

## 174. Απλό firewall με subnetting

Δίκτυο: LAN (192.168.1.0/24) → Router/Firewall → DMZ (10.0.0.0/24) → Internet.

## 175. Μέθοδος κρυπτογράφησης ενός κλειδιού (συμμετρική)

Χρήση ίδιου κλειδιού για κρυπτογράφηση και αποκρυπτογράφηση (AES, DES).

## 176. Firewall – βασικές λειτουργίες

Φιλτράρισμα πακέτων, NAT, stateful inspection. Linux: `iptables` ή `nftables`.

## 177. Αλγόριθμος Καίσαρα – αποκρυπτογράφηση

`Wklv lv qrw vhfxyh` → `This is not secure`

## 178. Χρήσεις SSL

HTTPS, email (SMTPS, IMAPS), VPN, VoIP.

## 179. INSERT INTO EMP (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM)

sql

```
INSERT INTO EMP VALUES (7369, 'ΣΑΛΑΜΟΥΡΑΣ', 'ΑΝΑΛΥΤΗΣ', 7902, TO_DATE('17-12-2004', 'DD-MM-YYYY'), 1900, 1000, NULL, NULL, NULL);
```

**180. UPDATE EMP SET ADDRESS='ΠΑΠΑΡΗΓΟΠΟΥΛΟΥ 53  
ΧΑΛΑΝΔΡΙ', ZIP='15231' WHERE EMPNO=7369;**

**181. Πράξεις σχέσεων (σχεσιακή άλγεβρα)**

Επιλογή, προβολή, συνένωση, διαίρεση, ένωση, τομή, καρτεσιανό γινόμενο.

**182. SELECT \* FROM EMP WHERE (JOB='Επ. Εργου' OR JOB='Γραμμ')  
AND DEPTNO=10;**

Επιστρέφει υπαλλήλους με JOB «Επ. Εργου» ή «Γραμμ» και τμήμα 10.

**183. SELECT \* FROM EMP WHERE SAL BETWEEN 1500 AND 1800;**

**184. DROP TABLE DEPT;**

**185. SELECT \* FROM EMP WHERE ENAME LIKE 'M%';**

**186. SELECT \* FROM EMP WHERE DEPTNO=30 ORDER BY SAL;**

**187. SELECT \* FROM EMP WHERE COMM > SAL;**

**188. SELECT \* FROM EMP WHERE SAL\*12 > 40000;**

**189. SELECT COUNT(\*) FROM EMP WHERE COMM IS NULL;**

**190. SELECT \* FROM EMP WHERE DEPTNO=30 ORDER BY COMM;**

**191. ALTER TABLE Dept DROP COLUMN LOC;**

**192. SELECT DISTINCT JOB FROM EMP;**

**193. SELECT \* FROM EMP WHERE JOB IN ('Γραμματέας', 'Αναλυτής',  
'Πωλητής');**

**194. SELECT DEPTNO, COUNT(DISTINCT JOB) FROM EMP GROUP BY DEPTNO;**

**195. SELECT \* FROM EMP WHERE SAL < 4000 AND SAL > 2000;**

**196. DELETE FROM EMP WHERE HIREDATE <= ADD\_MONTHS(SYSDATE, -25\*12) AND SAL > 3000;**

**197. DELETE FROM EMP WHERE HIREDATE <= ADD\_MONTHS(SYSDATE, -25\*12) AND DEPTNO IN (SELECT DEPTNO FROM DEPT WHERE LOC='Πώμη');**

**198. CREATE TABLE TEMP AS SELECT E.\*, D.DNAME FROM EMP E JOIN DEPT D ON E.DEPTNO = D.DEPTNO;**

**199. CREATE TABLE TEMP1 (ENAME, DNAME, SAL1) AS SELECT E.ENAME, D.DNAME, SAL+COMM FROM EMP E JOIN DEPT D ON E.DEPTNO = D.DEPTNO WHERE COMM IS NOT NULL;**

**200. UPDATE EMP SET SAL = SAL\*1.05 WHERE DEPTNO = (SELECT DEPTNO FROM DEPT WHERE DNAME='SALES') AND COMM IS NULL;**

**201. SELECT \* FROM EMP WHERE JOB='Πωλητής' AND DEPTNO=30 AND SAL BETWEEN 1500 AND 1500; (αν εννοείτε μόνο 1500)**

**202. SELECT ENAME, SAL, COMM, SAL+NVL(COMM,0) FROM EMP WHERE JOB='Πωλητής';**

**203. SELECT EXTRACT(YEAR FROM HIREDATE) AS YEAR, COUNT()  
FROM EMP GROUP BY EXTRACT(YEAR FROM HIREDATE) ORDER BY  
COUNT() DESC FETCH FIRST 1 ROW ONLY;**



**204. SELECT \* FROM EMP WHERE SAL > (SELECT MAX(SAL) FROM EMP WHERE ENAME IN ('ΑΛΕΒΙΖΟΣ','ΑΝΔΡΙΤΣΑΚΗΣ'));**

**205. SELECT MGR, COUNT(\*) FROM EMP WHERE MGR IS NOT NULL GROUP BY MGR;**

**206. SELECT ENAME, SAL, COMM FROM EMP WHERE COMM > 0.25\*SAL;**

**207. SELECT ENAME, SAL/25 AS DAILY, SAL/25/8 AS HOURLY FROM EMP WHERE DEPTNO=30;**

**208. SELECT DEPTNO, AVG(SAL) FROM EMP GROUP BY DEPTNO HAVING AVG(SAL) > 1000;**

**209. ALTER TABLE Dept ADD CONSTRAINT chk\_loc CHECK (LOC IN ('Αθήνα','Θεσσαλονίκη'));**

**210. DELETE FROM EMP WHERE DEPTNO=10;**

**211. SELECT ENAME, D.DNAME, COUNT(\*) OVER (PARTITION BY E.DEPTNO) AS TOTAL FROM EMP E JOIN DEPT D ON E.DEPTNO = D.DEPTNO;**

**212. Δημιουργία EMP και DEPT με constraints (δόθηκε παραπάνω).**

**213. SELECT \* FROM EMP WHERE DEPTNO = (SELECT DEPTNO FROM DEPT WHERE DNAME='Πωλήσεις') AND JOB != 'Πωλητής';**

**214. CREATE TABLE DEPT2 AS SELECT \* FROM DEPT; ALTER TABLE DEPT2 RENAME COLUMN JOB TO ERG;**

**215. Δημιουργία βάσης «BASE\_1»:**

sql

```
CREATE DATABASE BASE_1;
```

```
USE BASE_1;
```

```
SELECT DATABASE();
```

```
DROP DATABASE BASE_1;
```